

NETWORK TRAFFIC ANALYSIS USING WIRESHARK

A Practical SOC Analyst Portfolio Project

Prepared by

Ankita Saxena

PG Diploma in Cybersecurity

Aspiring SOC Analyst | Network Security Enthusiast | Wireshark Learner

GitHub: github.com/SaXen0Shield0WebGuard

Lab Environment: VMware Workstation • Kali Linux • Windows 10 • Wireshark

1. Project Overview

This report documents a hands-on network traffic analysis project carried out using **Wireshark** in a virtual lab environment. Traffic was generated between a Kali Linux machine, a Windows 10 machine, and external web servers, then captured and analyzed to understand how common network protocols behave in practice. The project focuses on identifying packet structures, protocol behavior, and communication patterns of the kind routinely encountered by SOC (Security Operations Center) analysts during traffic monitoring and investigation.

Objectives

- Capture live network traffic using Wireshark
- Analyze common networking protocols
- Understand packet flow and protocol behavior
- Learn TCP/IP communication in a practical setting
- Develop hands-on packet analysis skills
- Build a portfolio project suitable for entry-level SOC Analyst roles

2. Lab Environment & Tools

Lab Setup

Field	Detail
Hypervisor	VMware Workstation
Attacker / Analysis Machine	Kali Linux
Target Machine	Windows 10
Packet Analyzer	Wireshark
Network Type	NAT Network

Tools Used

- Wireshark – packet capture and protocol analysis
- Kali Linux – traffic generation and analysis platform
- Windows 10 – target / peer machine
- VMware Workstation – virtual lab hypervisor
- Ping & nslookup – command-line traffic generation utilities
- Firefox Browser – HTTP/HTTPS traffic generation

3. Protocols Analyzed

Seven core networking protocols were captured and analyzed in this lab. For each protocol, traffic was generated intentionally, captured live in Wireshark, and filtered using protocol-specific display filters to isolate relevant packets.

3.1 ICMP (Ping)

Purpose: Verify connectivity between Kali Linux and Windows 10.

- Successful Echo Request and Echo Reply exchanges
- No packet loss observed
- TTL = 128 (consistent with Windows default)
- Low latency between hosts

Wireshark Filter:

```
icmp
```

3.2 DNS

Purpose: Resolve domain names into IP addresses.

Generated using:

```
nslookup google.com
```

- DNS queries sent to the configured DNS server
- Multiple IPv4 and IPv6 addresses returned in the responses
- Communication observed over UDP Port 53

Wireshark Filter:

```
dns
```

3.3 HTTP

Purpose: Analyze unencrypted web traffic.

- HTTP GET request captured in plaintext
- HTTP/1.1 200 OK response received from the server
- Full request-response communication cycle observed

Wireshark Filter:

```
http
```

3.4 HTTPS / TLS

Purpose: Analyze encrypted web communication.

- Both TLS 1.2 and TLS 1.3 traffic captured
- Full TLS handshake observed (ClientHello, ServerHello, key exchange)

- Encrypted Application Data segments exchanged post-handshake
- Communication observed with Google servers

Wireshark Filter:

```
tls
```

3.5 TCP

Purpose: Study reliable communication and connection establishment.

The complete TCP Three-Way Handshake was captured:

```
SYN
↓
SYN-ACK
↓
ACK
```

- HTTP communication carried over the established TCP session
- Graceful connection termination observed (FIN, ACK)

Wireshark Filter:

```
tcp
```

3.6 UDP

Purpose: Analyze connectionless communication.

- DNS traffic observed carried over UDP
- Source Port used an ephemeral (dynamically assigned) port
- Destination Port fixed at 53 for DNS queries
- mDNS (multicast DNS) traffic also observed on the network

Wireshark Filter:

```
udp
```

3.7 ARP

Purpose: Understand IP-to-MAC address resolution.

- ARP Requests captured on the local network segment
- Corresponding ARP Replies observed
- ARP Probes and ARP Announcements identified

Example exchange:

```
Who has 192.168.32.2?
Tell 192.168.32.136
```

Wireshark Filter:

```
arp
```

4. Key Findings

- Successfully captured live network traffic across a multi-VM lab environment
- Verified end-to-end communication between virtual machines
- Analyzed seven common networking protocols in depth
- Observed and documented the full TCP Three-Way Handshake
- Studied encrypted HTTPS/TLS communication end-to-end
- Understood the complete DNS query and response process
- Learned ARP-based MAC address resolution on a local segment
- Practiced protocol-specific packet filtering and traffic analysis in Wireshark

5. Project Structure

The repository is organized as follows:

```
Network-Traffic-Analysis-Wireshark/  
|  
|-- README.md  
|-- Report/  
| |-- Network_Traffic_Analysis_Wireshark_Report.pdf  
|  
|-- Captures/  
| |-- icmp_capture.pcapng  
| |-- dns_capture.pcapng  
| |-- http_capture.pcapng  
| |-- https_tls_capture.pcapng  
| |-- tcp_capture.pcapng  
| |-- udp_capture.pcapng  
| |-- arp_capture.pcapng  
|  
|-- Screenshots/  
| |-- icmp.png  
| |-- dns.png  
| |-- http.png  
| |-- tls.png  
| |-- tcp.png  
| |-- udp.png  
| |-- arp.png  
|  
|-- LICENSE
```

6. Skills Demonstrated

Technical Skills

- Packet Capture & Packet Inspection
- TCP/IP Networking Fundamentals

- DNS, HTTP, and HTTPS/TLS Analysis
- TCP Three-Way Handshake Analysis
- UDP and ARP Analysis
- Network Troubleshooting
- Wireshark Display Filtering
- Cybersecurity Fundamentals

SOC Analyst Skills Gained

- Traffic Monitoring and Packet Analysis
- Protocol Identification
- Network Troubleshooting
- Basic Threat Investigation
- Understanding Client-Server Communication
- Log Interpretation
- Network Visibility

7. Future Improvements

- Analyze FTP traffic
- Analyze SSH traffic
- Analyze DHCP traffic
- Detect ARP Spoofing attacks
- Investigate DNS Tunneling techniques
- Perform Malware Traffic Analysis
- Export packet statistics and protocol hierarchy reports

8. References

- Wireshark Documentation
- RFC 791 – Internet Protocol (IP)
- RFC 792 – Internet Control Message Protocol (ICMP)
- RFC 793 – Transmission Control Protocol (TCP)
- RFC 768 – User Datagram Protocol (UDP)
- RFC 1035 – Domain Name System (DNS)
- RFC 826 – Address Resolution Protocol (ARP)

Author: Ankita Saxena | PG Diploma in Cybersecurity

Aspiring SOC Analyst | Network Security Enthusiast | Wireshark Learner

GitHub: github.com/SaXen0Shield0WebGuard